



Load Balancing for AI Inference

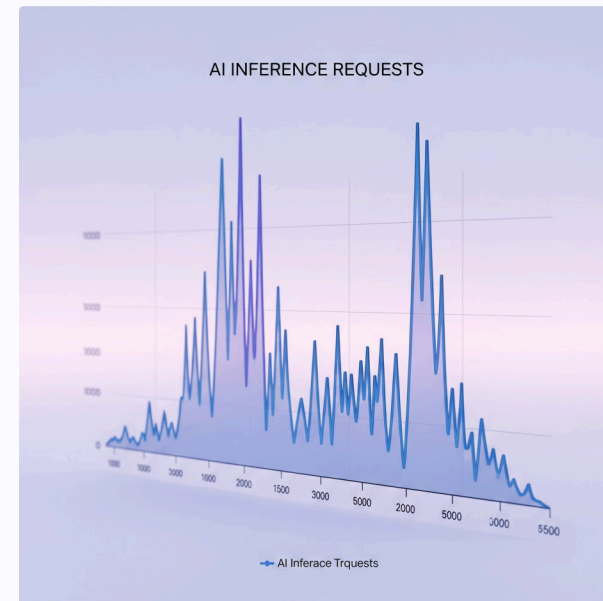
Keeping Latency Low and Throughput High

A technical deep dive for DevOps and ML infrastructure engineers managing production AI systems

Why Load Balancing Matters for AI

AI inference workloads present unique challenges that traditional load balancing approaches weren't designed to handle:

- Inference traffic exhibits **spiky, unpredictable patterns** that can overwhelm systems
- GPU backends have complex **batching and concurrency** requirements
- Poor load distribution creates **hot spots**, increases **tail latency**, and wastes expensive GPU resources



Goal: Smooth demand across resources, protect SLOs, and minimize infrastructure costs

Load Balancing Layers & Options

L4: TCP/UDP

- Google Maglev
- Linux IPVS
- Anycast routing
- Direct Server Return

Operates on transport layer with lower overhead but limited intelligence

L7: HTTP/HTTP2/gRPC

- Envoy Proxy
- NGINX
- HAProxy

Application-aware routing with richer request context for decision making

Global: Cross-Region

- DNS-based GSLB
- Global Anycast
- Multi-region orchestration

Directs traffic across geographic regions for resilience and latency optimization

In-cluster: Kubernetes

- K8s Service
- Ingress controllers
- Gateway API

Container-native routing with integrated service discovery and orchestration

The right LB approach depends on your specific infrastructure, scale requirements, and performance objectives

Core Load Balancing Algorithms



1

Round Robin / Weighted RR

Simple cyclic distribution for basic fairness. Weight factors adjust for heterogeneous backends.

2

Least Request / EWMA

Routes to least busy servers. EWMA (Exponentially Weighted Moving Average) factors in historical response times.

3

Consistent Hashing

Maps requests to backends based on key (session ID, user ID). Minimizes redistribution on cluster changes.

4

Priority / Weighted Pools

Directs traffic to preferred backends. Essential for canary deployments and blue/green rollouts.

GPU-Aware Considerations

Standard load balancing isn't enough for ML workloads. Your LB must understand GPU-specific characteristics:



Expose Concurrency Metrics

Route based on available GPU slots and queue depth, not just server health



Coordinate with Batching

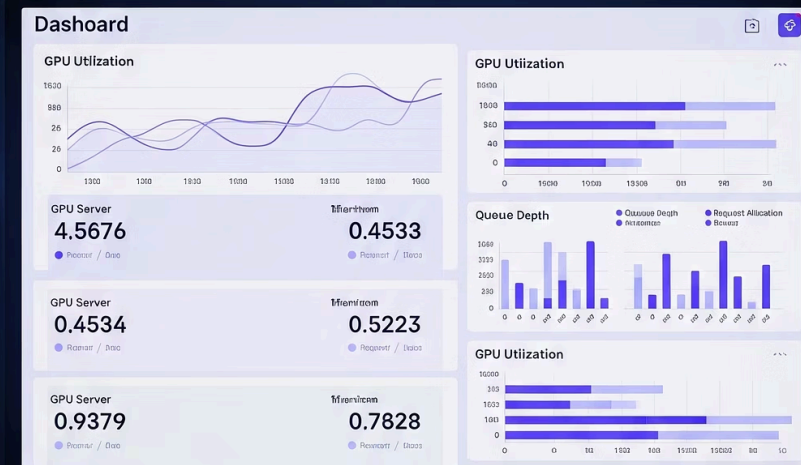
Align LB decisions with dynamic batching in servers like NVIDIA Triton



Handle GPU Heterogeneity

Apply appropriate weights when mixing different GPU types (A100, H100, etc.)

A GPU-aware load balancer must factor in the unique characteristics of accelerated computing to prevent bottlenecks



Health, Backpressure & Resilience

Circuit Breaking & Health Checks

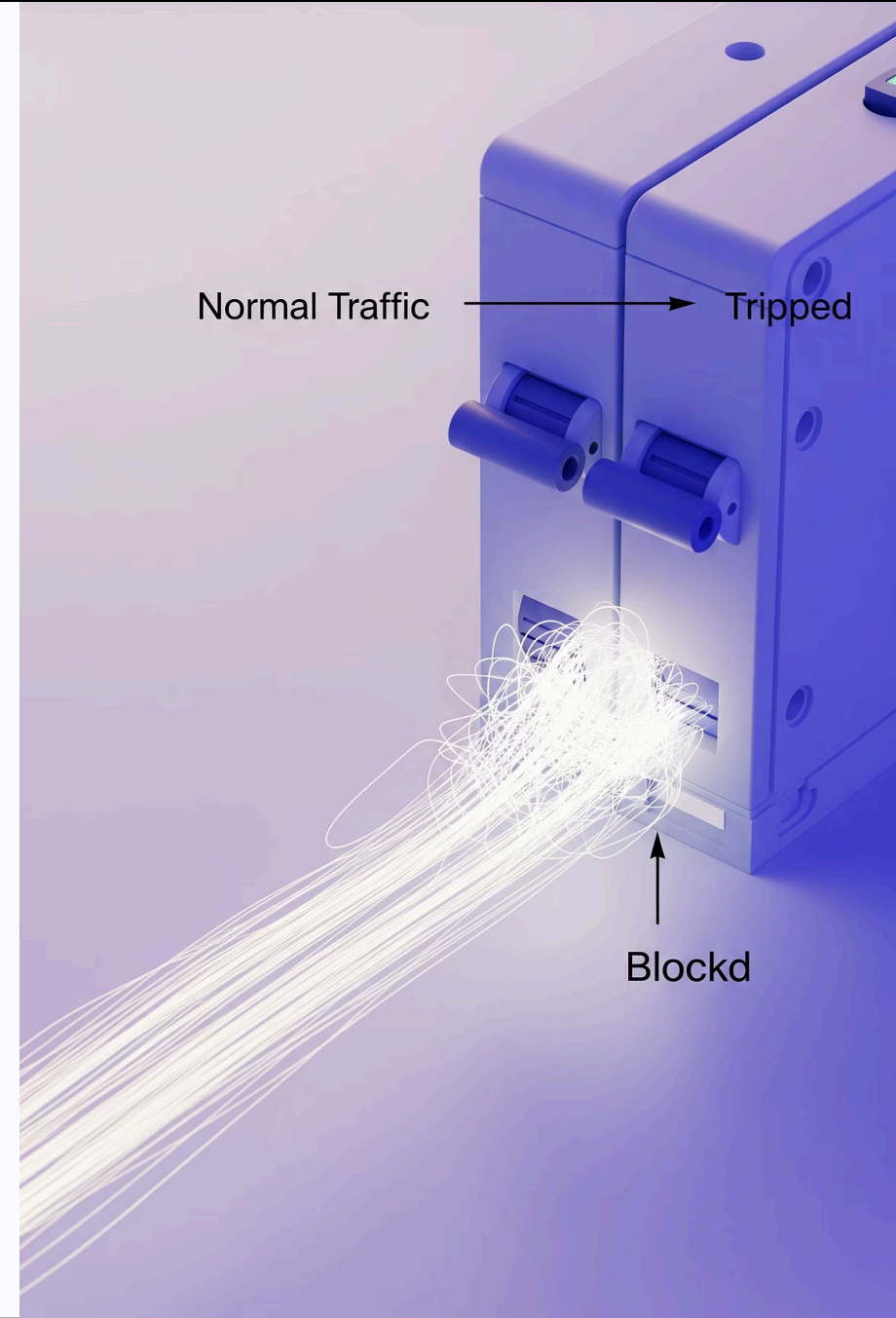
Implement multi-level monitoring:

- **L4 probes:** TCP connectivity checks
- **L7 probes:** Application-level health endpoints
- **Max pending:** Limit queue depth to prevent cascading failures
- **Max connections:** Cap concurrent requests per backend

Backpressure & Outlier Detection

Add safety mechanisms to handle pressure:

- **Outlier ejection:** Automatically remove unhealthy instances
- **Rate limiting:** Cap requests using token bucket algorithms
- **Gradual recovery:** Carefully reintroduce healing nodes
- **Retry budgets:** Limit retry attempts to prevent amplification



Sticky vs. Stateless Load Balancing

Prefer Stateless When Possible

Stateless requests enable true elasticity and simplify failover. Backend instances can be added/removed without affecting request routing.

Use Session Affinity Judiciously

Only implement stickiness when required (e.g., for model caching, session context). Document the performance vs. resilience tradeoff.

Smart Key Selection

If sticky routing is necessary, hash on user/idempotency keys rather than source IP which breaks with proxies and NAT.

Plan For Failures

Sticky sessions increase complexity during failovers. Implement proper fallback mechanisms when a preferred backend is unavailable.



Key Insight: The resiliency benefits of stateless routing typically outweigh the performance benefits of stickiness for most AI inference workloads.

Multi-Region Patterns



Active/Active

Run full capacity in multiple regions using anycast or latency-based DNS. Provides geographic redundancy and reduced latency for global users.



Smart Traffic Steering

Route based on multiple factors: client latency, regional capacity, and infrastructure costs. Dynamically adjust based on real-time conditions.



Regional Circuit Breakers

Implement regional isolation to contain failures. Prevent cascading issues across your global footprint while maintaining availability.

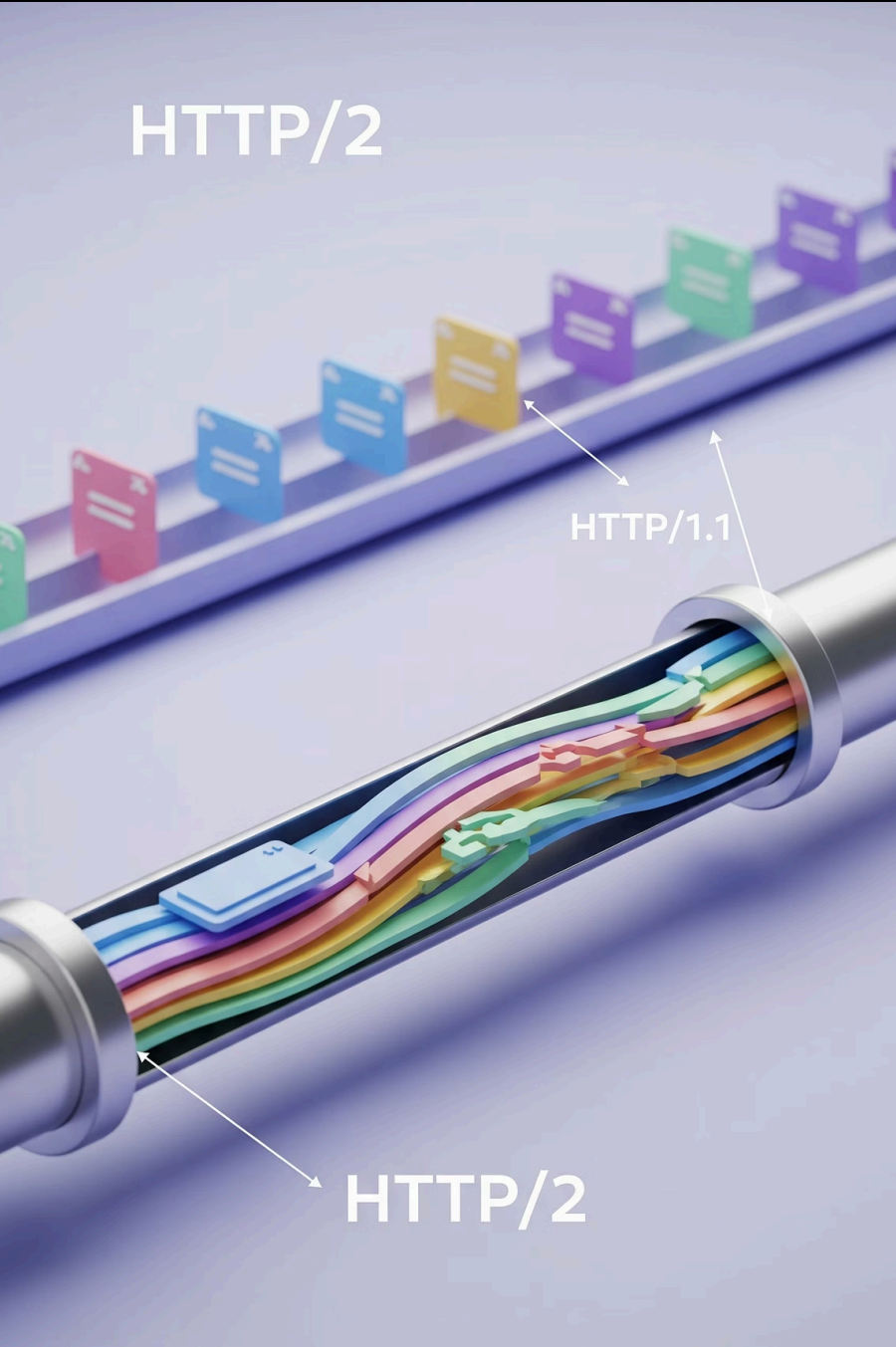


Failover Planning

Develop detailed runbooks for regional failures. Consider data locality constraints for models and ensure consistent deployment across regions.



HTTP/2



Protocol & Connection Tuning

Protocol Optimization

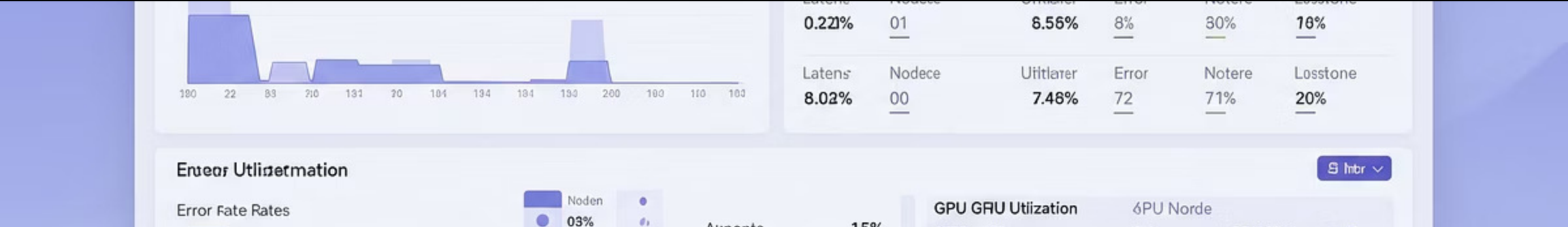
- **HTTP/2 & gRPC:** Leverage multiplexing for efficient long-lived connections
- **Compression:** Enable request/response compression for large payloads (model inputs/outputs)
- **Protocol Buffers:** Use compact binary serialization instead of verbose JSON when possible

Connection Parameters

- **Keepalive tuning:** Balance connection reuse with resource consumption
- **Max concurrent streams:** Configure appropriate limits for multiplexed connections
- **Timeout cascades:** Ensure client timeout > LB timeout > backend timeout
- **Warm pools:** Maintain minimum active connections to reduce cold start penalties



Pro Tip: For systems with highly variable inference times, implement per-request timeouts based on model complexity rather than global values.



Observability & SLOs



Latency Tracking

Monitor p50/p95/p99 latency distributions. Focus on tail latency for SLOs, not just averages which mask problems.



Tracing

Implement OpenTelemetry tracing across the entire request path from LB through model server for end-to-end visibility.



Queue Visibility

Export request queue time and backend utilization metrics. Separate LB queuing from actual inference time.

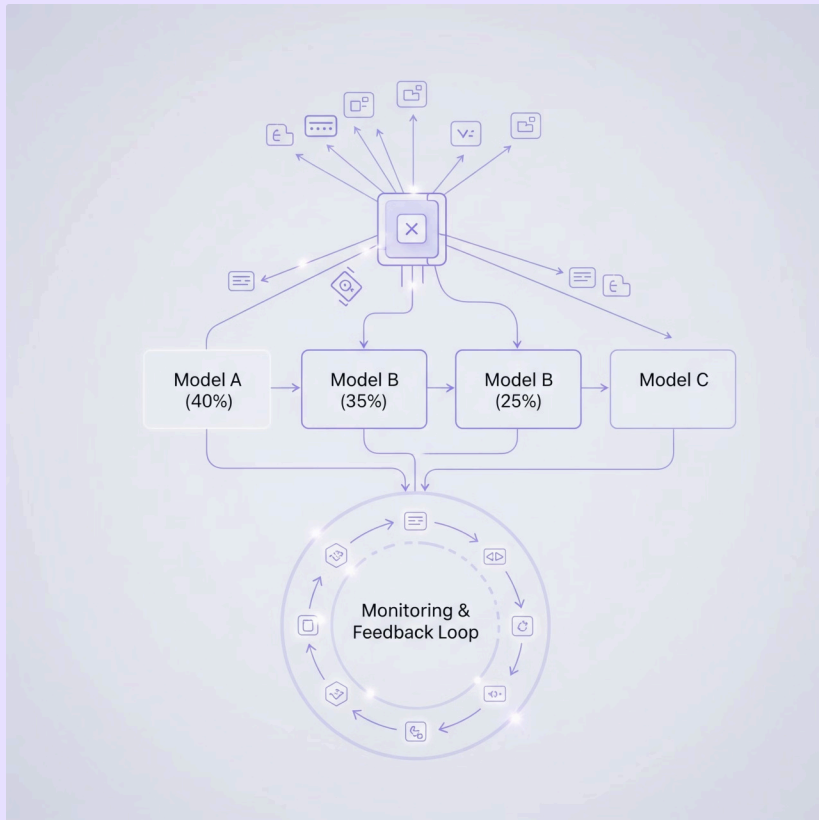


Dashboards & Probes

Create golden dashboards with synthetic probes that simulate real traffic patterns to detect issues before users do.

A comprehensive observability strategy allows you to quickly identify bottlenecks and optimize load balancing parameters based on real-world performance data

Canary, A/B Testing & Traffic Shadowing



Progressive deployment strategies reduce risk when rolling out new AI models or infrastructure changes

1

Canary Deployments

Route a small percentage of traffic to new model versions. Monitor closely and implement rapid rollback mechanisms if issues arise.

2

A/B Testing

Split traffic by user segment or request header to measure performance impact and business metrics across model variants.

3

Shadow Traffic

Duplicate production requests to new deployments without affecting user experience. Compare responses for correctness before live cutover.

4

SLO Gates

Establish clear performance thresholds that must be maintained during progressive rollouts. Automate rollback if SLOs are violated.

 **Implementation tip:** In Kubernetes environments, leverage Gateway API's TrafficSplit CRD for fine-grained traffic control during canary deployments.

Common Load Balancing Pitfalls

Thundering Herd

Problem: Mass reconnection attempts after deploy/scale-up overwhelm backends

Solution: Implement connection limits, gradual warmup, and jittered backoff

GPU Queue Blindness

Problem: Ignoring GPU queue depth leads to hotspotting despite server availability

Solution: Expose and utilize queue metrics from model servers in routing decisions

Connection Churn

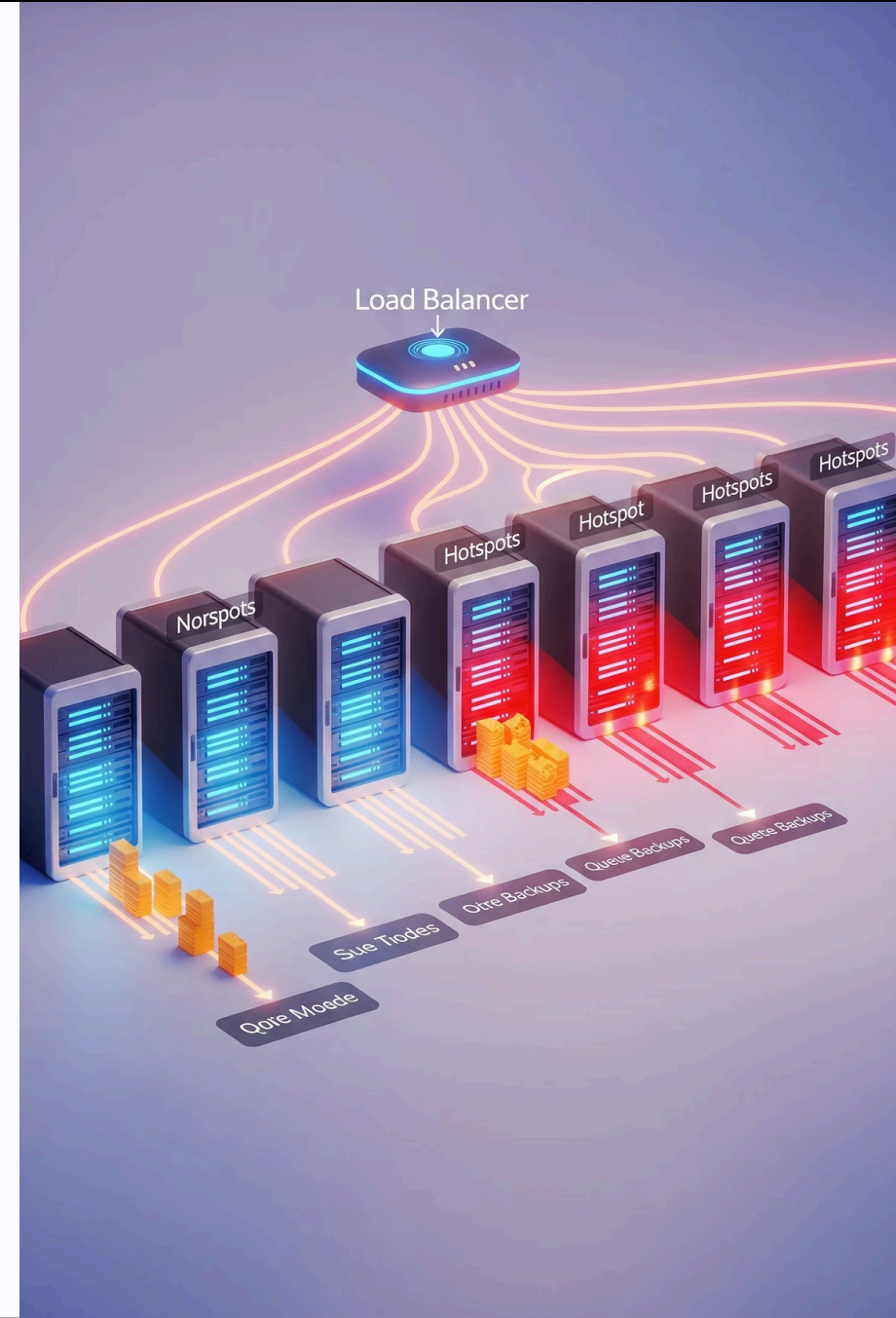
Problem: Lack of connection reuse creates head-of-line blocking and latency spikes

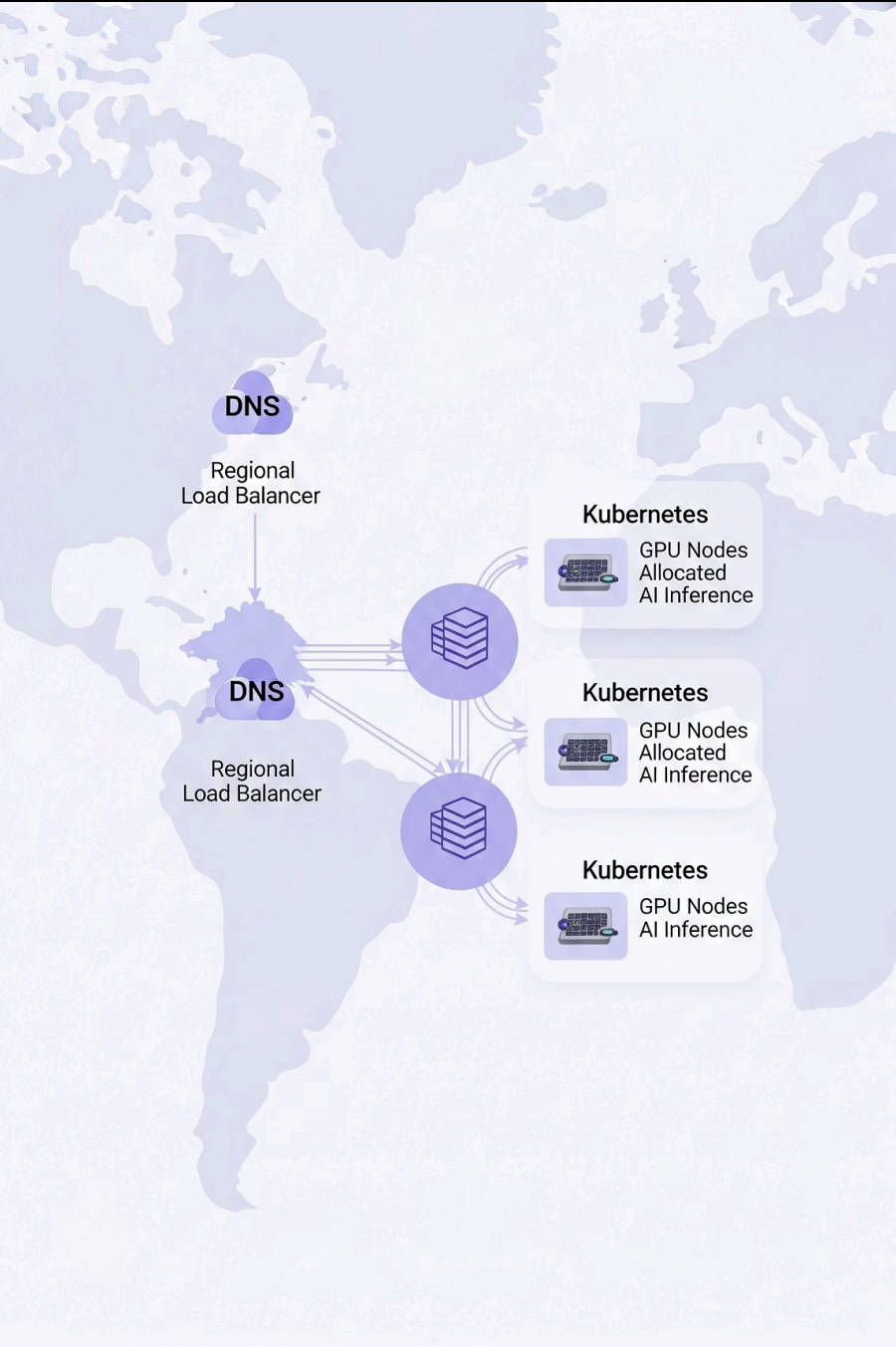
Solution: Use HTTP/2 or gRPC with proper keepalive and connection pool settings

Mixed GPU Tiers

Problem: Treating different GPU types equally leads to unpredictable tail latency

Solution: Apply appropriate weights based on benchmarked throughput capacity





Reference Architecture

Global Layer

- DNS with latency-based routing directs users to closest region
- Health checks automatically redirect from unhealthy regions
- Traffic policies adjust based on regional capacity

Edge Layer

- Anycast L4 load balancers distribute across regional fleets
- Regional L7 gateways (Envoy) handle protocol-aware routing
- TLS termination and authentication happens at this layer

Cluster Layer

- Kubernetes Gateway API directs to appropriate model services
- Specialized services (Triton/FastAPI) expose inference endpoints
- Pod anti-affinity ensures resilience to node failures

Feedback Loop

- Metrics collected from all layers feed into observability system
- Autoscalers adjust capacity based on current and predicted load
- Load balancer weights dynamically update based on backend health

Key Takeaways

Load balancing is critical for AI inference performance



Layer-Specific Approach

Choose load balancing solutions by layer (L4/L7/global) to match your specific infrastructure and performance goals.



GPU-Aware Routing

Make routing decisions based on GPU availability, queue depth, and batching capabilities to maximize expensive accelerator utilization.



Build in Resilience

Implement robust health checks, circuit breaking, and rate limiting to protect your system from cascading failures.



Measure & Iterate

Focus on tail latency metrics, not just averages. Continuously experiment and refine your load balancing strategy.

Questions? Contact the ML Infrastructure team